

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems **NAME:** K.SARANYA
COURSE CODE: CSA0501 **REG NO:** 192424409

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees	BL3 – Apply	5

	who work in the 'IT' department and earn more than 50000.		
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I K. SARANYA (192424409) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

		throughout			
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Note on Approach

Each solution below follows a consistent five-part structure: (1) restating the problem, (2) defining the sample relational schema and data used, (3) presenting the concept briefly, (4) the worked solution — relational algebra expression or SQL statement — with step-by-step reasoning, and (5) the resulting output table, so that the logic can be verified independently of the specific values used. Supporting diagrams (ER diagrams, Venn diagrams for joins, and an operator tree) are included where they aid understanding.

Question 1

Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

1.1 Schema and Sample Data

Two base relations are used throughout this solution:

EmpID	EName	Salary	DeptID	Designation
E01	Arun	45000	D01	Analyst
E02	Divya	62000	D02	Manager
E03	Karthik	38000	D01	Trainee
E04	Meena	71000	D03	Manager
E05	Rahul	55000	D02	Analyst

Employee(EmpID, EName, Salary, DeptID, Designation)

DeptID	DeptName	Location
--------	----------	----------

D01	IT	Chennai
D02	HR	Bengaluru
D03	Finance	Chennai

Department(DeptID, DeptName, Location)

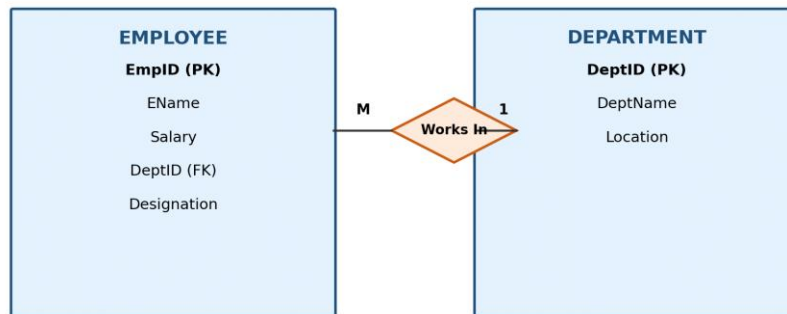


Figure 1.1 — ER relationship between EMPLOYEE and DEPARTMENT (M:1)

1.2 Selection (σ) — filter rows by condition

Retrieve employees earning above ₹50,000:

σ Salary > 50000 (Employee)				
EmpID	EName	Salary	DeptID	Designation
E02	Divya	62000	D02	Manager
E04	Meena	71000	D03	Manager
E05	Rahul	55000	D02	Analyst

1.3 Projection (π) — select specific columns

Retrieve only names and salaries (duplicates automatically eliminated):

π EName, Salary (Employee)	
EName	Salary
Arun	45000
Divya	62000
Karthik	38000
Meena	71000
Rahul	55000

1.4 Union ($\cup$) — combine two compatible relations

Let R1 = employees in IT (D01) and R2 = employees earning above 60000. Both are projected onto EmpID first so the schemas are union-compatible; the union gives everyone who is in IT OR earns above 60000:

$R1 = \pi \text{ EmpID } (\sigma \text{ DeptID} = 'D01' (\text{Employee})) \rightarrow \{E01, E03\}$

$$R2 = \pi \text{ EmpID } (\sigma \text{ Salary} > 60000 (\text{Employee})) \rightarrow \{E02, E04\}$$

$$R1 \cup R2 = \{E01, E02, E03, E04\}$$

1.5 Set Difference (−) — rows in one relation but not the other

Employees who are in IT but do NOT earn above 60000 ($R1 - R2$ using the sets above):

$$R1 - R2 = \{E01, E03\} \rightarrow \text{Arun, Karthik}$$

1.6 Cartesian Product (×) — pair every row of one relation with every row of the other

Employee × Department produces $|\text{Employee}| \times |\text{Department}| = 5 \times 3 = 15$ rows, pairing every employee with every department (unrelated pairs included). A representative excerpt:

Employee × Department				
EmpID	EName	Emp.DeptID	DeptID	DeptName
E01	Arun	D01	D01	IT
E01	Arun	D01	D02	HR
E01	Arun	D01	D03	Finance
E02	Divya	D02	D01	IT
...	...	...	...	...

1.7 Natural Join (⋈) — combine on the matching DeptID and eliminate the Cartesian noise

Employee ⋈ Department (equivalent to $\sigma \text{ Emp.DeptID} = \text{Dept.DeptID} (\text{Employee} \times \text{Department})$, duplicate column removed)

EmpID	EName	Salary	DeptID	DeptName	Location
E01	Arun	45000	D01	IT	Chennai
E02	Divya	62000	D02	HR	Bengaluru
E03	Karthik	38000	D01	IT	Chennai
E04	Meena	71000	D03	Finance	Chennai
E05	Rahul	55000	D02	HR	Bengaluru

This is the most commonly used operator in practice: it returns only matching rows (unlike ×, which returns all combinations), and unlike σ or π alone, it draws information from both relations at once.

Question 2

Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

2.1 Design Reasoning

- DEPARTMENT is created first because STUDENT references it via a foreign key — the referenced table must exist before the referencing table.
- DeptID is the PRIMARY KEY of DEPARTMENT and a FOREIGN KEY in STUDENT, enforcing referential integrity.

- NOT NULL is applied to mandatory fields; UNIQUE prevents duplicate department names; CHECK validates that DOB implies a realistic age; DEFAULT and AUTO-generation are avoided here to keep SID explicit and controlled by the application.

2.2 DDL Statements

```
CREATE TABLE DEPARTMENT (
    DeptID   VARCHAR(10) PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE STUDENT (
    SID      VARCHAR(10) PRIMARY KEY,
    Name     VARCHAR(50) NOT NULL,
    DOB      DATE      NOT NULL CHECK (DOB <= '2010-01-01'),
    DeptID   VARCHAR(10),
    CONSTRAINT fk_student_dept FOREIGN KEY (DeptID)
        REFERENCES DEPARTMENT(DeptID)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
```

2.3 Constraint Summary

Constraint	Applied On	Purpose
PRIMARY KEY	DeptID, SID	Uniquely identifies each row; disallows NULLs
FOREIGN KEY	STUDENT.DeptID	Ensures every student belongs to an existing department
NOT NULL	DeptName, Name, DOB	Mandatory business fields cannot be left empty
UNIQUE	DeptName	Prevents two departments from sharing a name
CHECK	DOB	Rejects biologically invalid / future birth dates
ON DELETE SET NULL	STUDENT.DeptID	If a department is removed, its students are not deleted, only unlinked

Question 3

Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

3.1 Sample Sales Dataset

SaleID	Product	Category	Region	Qty	Price
S1	Wheat Seeds	Seeds	South	40	250
S2	Urea	Fertilizer	South	60	300
S3	Rice Seeds	Seeds	North	25	280
S4	DAP	Fertilizer	North	35	450
S5	Pesticide-A	Pesticide	South	15	600
S6	Rice Seeds	Seeds	South	50	280
S7	Urea	Fertilizer	West	20	300

Sales(SaleID, Product, Category, Region, Qty, Price)

3.2 Query A — Total revenue per category (SUM + GROUP BY)

```
SELECT Category, SUM(Qty * Price) AS TotalRevenue
FROM Sales
GROUP BY Category;
```

Category	TotalRevenue
Seeds	35,500
Fertilizer	39,750
Pesticide	9,000

3.3 Query B — Categories with more than one product sold, avg price above 260 (HAVING)

```
SELECT Category, COUNT(*) AS NumSales, AVG(Price) AS AvgPrice,
       MAX(Price) AS MaxPrice, MIN(Price) AS MinPrice
FROM Sales
GROUP BY Category
HAVING COUNT(*) > 1 AND AVG(Price) > 260;
```

Category	NumSales	AvgPrice	MaxPrice	MinPrice
Seeds	3	270	280	250
Fertilizer	3	350	450	300

Note the distinction: WHERE filters individual rows before grouping, whereas HAVING filters entire groups after aggregation — Pesticide is excluded here because its COUNT(*) of 1 fails the HAVING condition, not because any single row was invalid.

3.4 Query C — Region-wise sales count

```
SELECT Region, COUNT(*)
AS NumTransactions FROM
Sales GROUP BY Region;
```

Region	NumTransactions
South	4
North	2
West	1

Question 4

Write a correlated sub-query to find employees earning more than the average salary of their own department.

4.1 Concept

A correlated sub-query references a column from the outer query, so it cannot be evaluated once — it is logically re-evaluated for every row the outer query considers. Here, for each employee, the inner query computes the average salary of only that employee's own department (via the WHERE e2.DeptID = e1.DeptID link), then the outer query keeps the employee if their salary beats that departmental average.

4.2 Sample Data

EmpID	EName	DeptID	Salary
E01	Arun	D01	45000
E02	Divya	D02	62000
E03	Karthik	D01	38000
E04	Meena	D03	71000
E05	Rahul	D02	55000
E06	Sneha	D01	52000

4.3 SQL Solution

```
SELECT e1.EmpID, e1.EName, e1.DeptID, e1.Salary
FROM Employee e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employee e2
    WHERE e2.DeptID = e1.DeptID -- correlation condition
)
ORDER BY e1.DeptID;
```

4.4 Execution Trace

DeptID	Dept. Avg Salary (computed per row)	Qualifying Employee(s)
D01	$(45000+38000+52000)/3 = 45,000$	Sneha (52000 > 45000)
D02	$(62000+55000)/2 = 58,500$	Divya (62000 > 58500)
D03	$71000/1 = 71,000$	None (Meena is not > her own average)

4.5 Final Output

EmpID	EName	DeptID	Salary
E06	Sneha	D01	52000
E02	Divya	D02	62000

Question 5

Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

5.1 Sample Data

EmpID	EName
E01	Arun
E02	Divya
E03	Karthik

Employee(EmpID, EName)

ProjID	PName	EmpID
P1	Crop Advisory	E01
P2	Weather Alert	E01

P3	Market Price Bot	E04
----	------------------	-----

Project(ProjID, PName, EmpID) — note EmpID = E04 does not exist in Employee, and E02/E03 have no projects; this deliberately exercises every join type.



Figure 5.1 — Visual comparison of the five join types used below

5.2 INNER JOIN — only matching rows

SELECT e.ENAME, p.PName FROM Employee e INNER JOIN Project p ON e.EmpID = p.EmpID;	
EName	PName
Arun	Crop Advisory
Arun	Weather Alert

5.3 LEFT OUTER JOIN — all employees, matched projects where they exist

SELECT e.ENAME, p.PName FROM Employee e LEFT JOIN Project p ON e.EmpID = p.EmpID;	
EName	PName
Arun	Crop Advisory
Arun	Weather Alert
Divya	NULL
Karthik	NULL

5.4 RIGHT OUTER JOIN — all projects, matched employees where they exist

SELECT e.ENAME, p.PName FROM Employee e RIGHT JOIN Project p ON e.EmpID = p.EmpID;	
EName	PName
Arun	Crop Advisory
Arun	Weather Alert
NULL	Market Price Bot

5.5 FULL OUTER JOIN — every employee and every project, matched or not

SELECT e.ENAME, p.PName FROM Employee e FULL OUTER JOIN Project p ON e.EmpID = p.EmpID;	
EName	PName
Arun	Crop Advisory
Arun	Weather Alert
Divya	NULL
Karthik	NULL
NULL	Market Price Bot

5.6 CROSS JOIN — Cartesian product (3 employees × 3 projects = 9 rows)

SELECT e.ENAME, p.PName FROM Employee e CROSS JOIN Project p;	
---	--

Produces every possible (employee, project) pairing regardless of assignment — useful chiefly as a base for generating combinations, not for reporting real assignments.

Question 6

Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

6.1 Why a View

A view stores a SELECT statement, not data. Every time it is queried, the underlying tables are re-evaluated, so the numbers stay live as Employee/Department rows change. This is ideal for a recurring HR report.

6.2 View Definition

```
CREATE VIEW DeptSalaryView AS
SELECT d.DeptName,
       COUNT(e.EmpID) AS TotalEmployees,
       AVG(e.Salary) AS AvgSalary
FROM Department d
LEFT JOIN Employee e ON d.DeptID = e.DeptID
GROUP BY d.DeptName;
```

6.3 Resulting View Contents (using the Q1 dataset)

DeptName	TotalEmployees	AvgSalary
IT	2	41,500
HR	2	58,500
Finance	1	71,000

6.4 Querying the View

Departments whose average salary exceeds 50,000:

SELECT * FROM DeptSalaryView WHERE AvgSalary > 50000;		
DeptName	TotalEmployees	AvgSalary
HR	2	58,500
Finance	1	71,000

The consuming query above treats DeptSalaryView exactly like a table — it is unaware that a join and a GROUP BY are happening underneath, which is precisely the abstraction benefit a view provides.

Question 7

Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

7.1 Why Division

This is the classic "for all" query. Relational algebra's division operator ($\div$) is built exactly for this shape of problem: given Enrollment(SID, CID) and the full set of Course(CID), $R \div S$ returns every SID that is paired with every CID in S — i.e., students missing even one course are excluded.

7.2 Sample Data

SID	CID
S1	C1
S1	C2
S1	C3
S2	C1
S2	C2
S3	C1
S3	C2
S3	C3

Enrollment(SID, CID) | Course offered = {C1, C2, C3}

7.3 Expression

Result = π SName (Student $\bowtie$ (π SID,CID(Enrollment) $\div$ π CID(Course)))

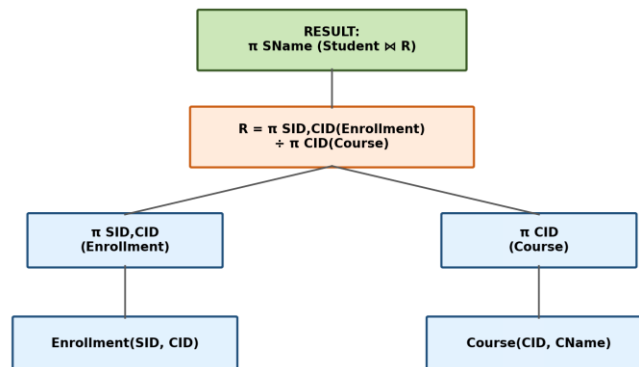


Figure 7.1 — Operator tree: division is evaluated first, then joined back to Student for names

7.4 Expressing $\div$ Using Primitive Operators ($\times$ and $-$)

Because $\div$ is not a primitive operator, a DBMS engine (and an exam answer, for full rigor) expresses it as:

T1 = π SID (Enrollment)
 T2 = (T1 $\times$ Course) - π SID,CID (Enrollment)
 T3 = π SID (T2)

$R = T1 - T3$ -- students with NO missing course

Intuitively: $T1 \times \text{Course}$ lists every (student, course) pair that SHOULD exist if that student took everything; subtracting the actual Enrollment leaves only the pairs a student is missing; $T3$ is therefore the set of students missing at least one course; the final subtraction removes them, leaving only students enrolled in every course.

7.5 Result

Applying this to the sample data: S1 and S3 are enrolled in all three courses; S2 is missing C3.

SID	SName
S1	Student S1
S3	Student S3

Question 8

Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

8.1 Scenario

Using Employee(EmpID PK, EName, Salary, DeptID FK → Department) and Department(DeptID PK, DeptName), demonstrate each DML statement together with the constraint that governs it.

8.2 INSERT — enforced by PRIMARY KEY and FOREIGN KEY

```
INSERT INTO Employee (EmpID, EName, Salary, DeptID)
VALUES ('E07', 'Priya', 48000, 'D01');
-- Succeeds: 'E07' is unique (PK) and 'D01' exists in Department (FK)
```

```
INSERT INTO Employee (EmpID, EName, Salary, DeptID)
VALUES ('E01', 'Dup', 30000, 'D01');
-- Fails: violates PRIMARY KEY — 'E01' already exists
```

```
INSERT INTO Employee (EmpID, EName, Salary, DeptID)
VALUES ('E08', 'Ghost', 40000, 'D99');
-- Fails: violates FOREIGN KEY — 'D99' is not a valid DeptID
```

8.3 UPDATE — enforced by CHECK and FOREIGN KEY

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE DeptID = 'D01';
-- Succeeds if a CHECK (Salary > 0) constraint exists and remains satisfied
```

```
UPDATE Employee SET DeptID = 'D99' WHERE EmpID = 'E07';
-- Fails: 'D99' does not exist in Department (FOREIGN KEY violation)
```

8.4 DELETE — enforced by referential integrity rule (RESTRICT / CASCADE)

```
DELETE FROM Employee WHERE EmpID = 'E07';
-- Always safe: Employee is the 'many' side, nothing else references it

DELETE FROM Department WHERE DeptID = 'D01';
-- Behaviour depends on the FK's ON DELETE clause:
-- ON DELETE RESTRICT -> fails while E01/E03/E06 still reference D01
-- ON DELETE CASCADE -> silently deletes those employees too
-- ON DELETE SET NULL -> keeps the employees, sets their DeptID to NULL
```

8.5 Summary

Statement	Constraint Enforced	Effect on Failure
INSERT	PRIMARY KEY, FOREIGN KEY, NOT NULL	Row rejected, transaction rolled back
UPDATE	CHECK, FOREIGN KEY	New value rejected, row unchanged
DELETE	Referential integrity (RESTRICT/CASCADE/SET NULL)	Depends on declared ON DELETE rule

Question 9

Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

9.1 Concept

Tuple Relational Calculus (TRC) is declarative: instead of specifying operations (as in relational algebra), it specifies the properties the desired tuples must satisfy, in the form $\{ t \mid P(t) \}$, read as "the set of all tuples t such that predicate $P(t)$ holds."

9.2 Expression

```
{ t | t ∈ Employee ∧ t.DeptID = 'D01' ∧ t.Salary > 50000 }
```

9.3 Reading the Expression

Component	Meaning
t	A tuple variable ranging over rows
$t \in \text{Employee}$	t is restricted to the Employee relation
$t.\text{DeptID} = \text{'D01'}$	Keep only rows from the IT department
$t.\text{Salary} > 50000$	Keep only rows whose salary exceeds 50,000
$\wedge$	Logical AND — both conditions must hold simultaneously

9.4 Equivalent SQL (for cross-verification)

```
SELECT * FROM Employee WHERE DeptID = 'D01' AND Salary > 50000;
```

9.5 Applying to Sample Data (from Q4)

EmpID	EName	DeptID	Salary
E06	Sneha	D01	52000

Only Sneha satisfies both predicates: she belongs to D01 (IT) and her salary of 52,000 exceeds 50,000. Arun (45000) and Karthik (38000) are excluded by the salary predicate despite being in the correct department.

Question 10

Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

10.1 Relational Schema

Relation	Attributes
PASSENGER	PassID (PK), Name, Passport, Phone
AIRCRAFT	AircraftID (PK), Model, Capacity
FLIGHT	FlightID (PK), FlightNo, AircraftID (FK), Fare
AIRPORT	AirportID (PK), City, Code
SCHEDULE	SchedID (PK), FlightID (FK), AirportID (FK), DepTime, ArrTime
RESERVATION	ResID (PK), PassID (FK), FlightID (FK), SeatNo, Status
PAYMENT	PaymentID (PK), ResID (FK), Amount, Mode

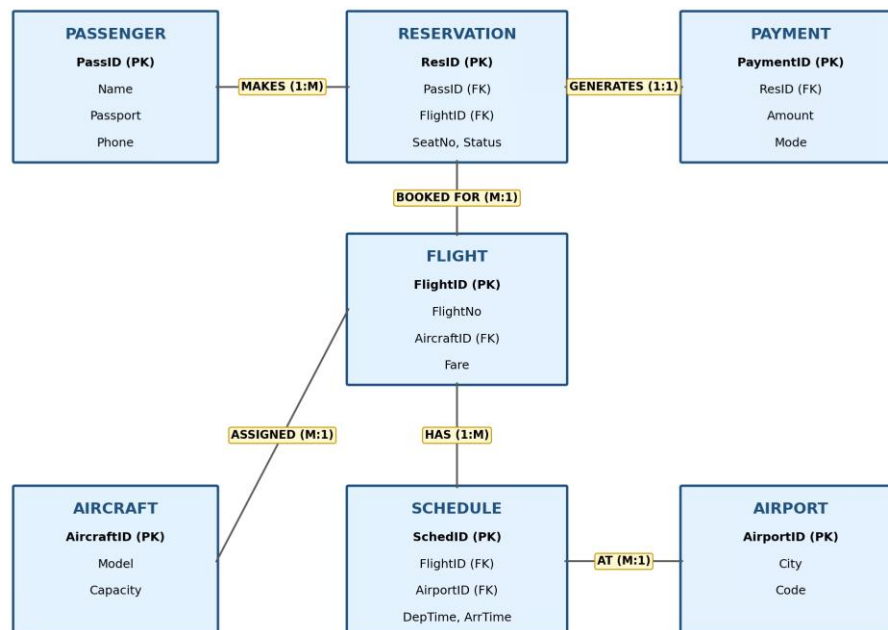


Figure 10.1 — ER diagram of the Airline Reservation System

10.2 Query 1 — JOIN: passenger name with their booked flight number and fare

```
SELECT p.Name, f.FlightNo, f.Fare, r.Status
FROM Reservation r
JOIN Passenger p ON r.PassID = p.PassID
JOIN Flight f ON r.FlightID = f.FlightID;
```

10.3 Query 2 — Multi-table JOIN: full itinerary with departure/arrival airports

```
SELECT p.Name, f.FlightNo, a1.City AS DepCity, a2.City AS ArrCity, s.DepTime
FROM Reservation r
JOIN Passenger p ON r.PassID = p.PassID
JOIN Flight f ON r.FlightID = f.FlightID
JOIN Schedule s ON f.FlightID = s.FlightID
JOIN Airport a1 ON s.AirportID = a1.AirportID
JOIN Airport a2 ON f.FlightID = f.FlightID; -- arrival leg modelled via a second Schedule row
```

10.4 Query 3 — Sub-query: passengers who paid more than the average fare paid

```
SELECT p.Name, pay.Amount
FROM Payment pay
JOIN Reservation r ON pay.ResID = r.ResID
JOIN Passenger p ON r.PassID = p.PassID
WHERE pay.Amount > (SELECT AVG(Amount) FROM Payment);
```

10.5 Query 4 — Sub-query with NOT IN: flights that have never been booked

```
SELECT FlightNo
FROM Flight
WHERE FlightID NOT IN (SELECT DISTINCT FlightID FROM Reservation);
```

10.6 Query 5 — View: daily revenue summary, then queried

```
CREATE VIEW DailyRevenue AS
SELECT CAST(s.DepTime AS DATE) AS FlightDate, SUM(pay.Amount) AS Revenue
FROM Payment pay
JOIN Reservation r ON pay.ResID = r.ResID
JOIN Schedule s ON r.FlightID = s.FlightID
GROUP BY CAST(s.DepTime AS DATE);

SELECT * FROM DailyRevenue WHERE Revenue > 100000 ORDER BY FlightDate;
```

Together, these five queries demonstrate join composition (Q1–Q2), scalar and set sub-queries (Q3–Q4), and view-based reporting (Q5) — covering the full range of retrieval techniques requested for the airline schema.